1. Docker란 무엇인가? 🐋

> **학습 목표**: Docker의 기본 개념과 필요성을 이해하고, 컨테이너 기술이 소프트웨어 개발과 배포에서 어떤 문제를 해결하는지 설명할 수 있습니다.

⏱️ **예상 학습 시간**: 1-2시간
난이도: ⭐ ☆☆☆☆ (1개/5개)

목차

- 1.1 Docker 기본 개념
- 1.2 왜 Docker를 사용하나?
- 1.3 핵심 용어 설명
- 1.4 실생활 비유로 이해하기

1.1 Docker 기본 개념

🎯 한 줄 정의

Docker는 애플리케이션을 "컨테이너"라는 표준화된 유닛으로 패키징하여 어디서든 동일하게 실행할 수 있게 해주는 오픈소스 플랫폼입니다.

📖 초보자를 위한 쉬운 설명

여러분이 프로그램을 만들었다고 상상해봅시다. 이 프로그램은 여러분의 컴퓨터에서는 완벽하게 작동합니다. 하지만 팀원의 컴퓨터나 서버에 설치하려고 하면 다음과 같은 문제들이 발생합니다:

```
개발자: "제 컴퓨터에서는 잘 되는데요?"

문제 1: Java 버전이 달라요
- 내 컴퓨터: Java 11
- 서버: Java 8
- 결과: 프로그램이 실행되지 않음 ❌

문제 2: 라이브러리가 없어요
- 내 컴퓨터: PostgreSQL 13 설치됨
- 팀원 컴퓨터: PostgreSQL 설치 안 됨
- 결과: 데이터베이스 연결 실패 ❌
```

```
문제 3: 운영체제가 달라요
- 내 컴퓨터: Windows
- 서버: Linux
- 결과: 경로 문제, 권한 문제 발생 ❌
```

Docker는 이 모든 문제를 해결합니다!

Docker를 사용하면:

1. **프로그램**과
2. **프로그램이 실행되는 데 필요한 모든 것**(Java, 라이브러리, 설정 파일 등)을
3. **하나의 패키지(컨테이너)**로 묶습니다

이렇게 만든 컨테이너는 **어떤 컴퓨터에서든 똑같이 실행**됩니다!

🎁 Docker = 배송용 컨테이너

실제 화물 운송을 생각해봅시다:

🚫 컨테이너가 없던 시대 (1950년대 이전)

```
[바나나 상자] [TV 박스] [옷가방] [가구]
         ↓
각각 다른 크기, 다른 포장, 다른 운송 방법
         ↓
배에 싣기 어렵고, 내리기 어렵고, 관리가 복잡함
```

✅ 표준 컨테이너 등장 후 (1956년~)

```
┌──────────┐ ┌──────────┐ ┌──────────┐
│ 바나나   │ │   TV     │ │   옷     │
│          │ │          │ │          │
└──────────┘ └──────────┘ └──────────┘
표준 크기 컨테이너로 통일
         ↓
배, 기차, 트럭 어디서든 동일하게 운송 가능
         ↓
물류 혁명! 운송비 95% 감소
```

Docker도 똑같은 원리입니다!

```
소프트웨어의 "표준 컨테이너"
```

```
┌─────────────────────┐  ┌─────────────────────┐
│   웹 애플리케이션    │  │    데이터베이스      │
│   + Java 11         │  │    + PostgreSQL     │
│   + 라이브러리       │  │    + 설정 파일       │
└─────────────────────┘  └─────────────────────┘

          ↓
어느 컴퓨터에서든 동일하게 실행
(Windows, Mac, Linux, AWS, Azure...)
```

💡 Docker의 핵심 원리

1️⃣ 격리 (Isolation)

각 컨테이너는 독립적인 환경에서 실행됩니다.

```
내 컴퓨터
├── 🐳 컨테이너 A (Java 8 + MySQL 5.7)
│    └── 프로젝트 A 실행 중
│
├── 🐳 컨테이너 B (Java 11 + PostgreSQL 13)
│    └── 프로젝트 B 실행 중
│
└── 🐳 컨테이너 C (Python 3.9 + Redis)
     └── 프로젝트 C 실행 중

각 컨테이너는 서로 간섭하지 않음!
```

2️⃣ 이식성 (Portability)

한 번 만든 컨테이너는 어디서든 실행됩니다.

```
개발자 노트북 (Windows)
     ↓ 컨테이너 생성
Docker Hub에 업로드
     ↓ 컨테이너 다운로드
회사 서버 (Linux)
     ↓ 동일한 컨테이너 실행
클라우드 (AWS/Azure/GCP)
     ↓ 여전히 동일하게 작동!
```

3️⃣ 경량성 (Lightweight)

가상머신(VM)보다 훨씬 가볍고 빠릅니다.

```
전통적인 가상머신 (VM)

┌─────────────────────────────────┐
│  App A  │  App B  │  App C  │
├─────────┼─────────┼─────────┤
│ Guest OS│ Guest OS│ Guest OS│  각각 수 GB
├─────────┴─────────┴─────────┤
│      Hypervisor             │
├─────────────────────────────┤
│      Host OS                │
├─────────────────────────────┤
│      Hardware               │
└─────────────────────────────┘

부팅 시간: 수 분
크기: 수십 GB

Docker 컨테이너

┌─────────────────────────────┐
│  App A  │  App B  │  App C  │  각각 수십 MB
├─────────┴─────────┴─────────┤
│      Docker Engine          │
├─────────────────────────────┤
│      Host OS                │
├─────────────────────────────┤
│      Hardware               │
└─────────────────────────────┘

시작 시간: 수 초
크기: 수백 MB
```

왜 이렇게 차이가 날까요?

- **가상머신**: 전체 운영체제를 포함 (Windows 전체를 복사하는 것과 같음)
- **Docker**: 운영체제를 공유하고 애플리케이션만 격리 (필요한 부분만 가져옴)

1.2 왜 Docker를 사용하나?

🎯 실제 개발 시나리오로 이해하기

시나리오 1: 신입 개발자 온보딩

🚫 Docker 없이 (전통적인 방법)

```
1일차 – 개발 환경 설정
09:00  Java JDK 11 다운로드 및 설치
```

```
09:30  환경 변수 설정 (PATH 추가)
10:00  오류 발생: "JAVA_HOME이 설정되지 않았습니다"
10:30  구글링하며 문제 해결
11:00  PostgreSQL 설치 시작
11:30  포트 충돌 발생 (이미 5432 사용 중)
12:00  점심 식사
13:00  PostgreSQL 재설치
13:30  Gradle 설치
14:00  프로젝트 클론 및 빌드 시도
14:30  빌드 실패: "라이브러리 버전 불일치"
15:00  라이브러리 버전 맞추기
16:00  Redis 설치 시작
16:30  설정 파일 찾기
17:00  또 다른 오류...
17:30  퇴근 (개발 환경 설정 미완료)

결과: 하루 종일 설정만 하고 코드는 한 줄도 못 씀 😭
```

✅ Docker 사용

```
09:00  프로젝트 클론
       git clone https://github.com/myproject.git

09:05  Docker Compose 실행
       docker-compose up -d

09:08  모든 서비스 실행 완료!
       ✅ PostgreSQL 실행 중
       ✅ Redis 실행 중
       ✅ Kafka 실행 중
       ✅ 애플리케이션 실행 중

09:10  개발 시작!
       코드 작성, 테스트, 커밋

결과: 10분 만에 개발 시작 🚀
```

시나리오 2: "내 컴퓨터에서는 되는데요?" 문제

🚫 실제로 자주 발생하는 상황

```
개발자 A (Windows)
- Node.js 14.x
- PostgreSQL 13
- 로컬에서 완벽하게 작동 ✅
```

```
개발자 B (Mac)
- Node.js 16.x
- PostgreSQL 14
- 실행 안 됨: "문법 오류" ❌

서버 (Linux)
- Node.js 12.x
- PostgreSQL 12
- 실행 안 됨: "연결 오류" ❌

회의실 대화:
개발자 A: "제 컴퓨터에서는 완벽하게 돌아가는데요?"
개발자 B: "저는 안 되는데요? 뭐가 문제죠?"
팀장: "서버에도 배포가 안 되네요..."
```

✅ Docker 사용 시

```
모든 환경에서 동일한 컨테이너 사용

개발자 A (Windows)
docker run myapp:1.0
→ 실행 성공 ✅

개발자 B (Mac)
docker run myapp:1.0
→ 실행 성공 ✅

서버 (Linux)
docker run myapp:1.0
→ 실행 성공 ✅

회의실 대화:
팀장: "모든 환경에서 동일하게 작동하네요!"
개발자들: "Docker 최고!" 👍
```

📊 Docker의 실제 장점

1️⃣ 개발 생산성 향상

시간 절약

```
전통적인 방법:
- 개발 환경 설정: 4~8시간
- 새로운 팀원 온보딩: 1~2일
```

```
- 다른 프로젝트로 전환: 2~4시간

Docker 사용 시:
- 개발 환경 설정: 5~10분
- 새로운 팀원 온보딩: 10~15분
- 다른 프로젝트로 전환: 1~2분

시간 절약: 약 90%! ⚡
```

일관성 보장

```
✅ 모든 팀원이 동일한 환경에서 작업
✅ 개발/테스트/운영 환경 동일
✅ "내 컴퓨터에서는 되는데요?" 문제 사라짐
```

2️⃣ 복잡한 환경도 간단하게

마이크로서비스 아키텍처 예시

```
전통적인 방법:
1. PostgreSQL 설치 및 설정 (30분)
2. Redis 설치 및 설정 (20분)
3. Kafka 설치 및 설정 (1시간)
4. Zookeeper 설정 (30분)
5. Elasticsearch 설치 (40분)
6. Nginx 설정 (30분)
7. 각 서비스 간 네트워크 설정 (1시간)
8. 포트 충돌 해결 (???)
9. 권한 문제 해결 (???)
총 소요 시간: 최소 4~5시간

Docker 사용:
docker-compose.yml 작성 (10분)
docker-compose up -d (실행 2분)
총 소요 시간: 12분
```

실제 docker-compose.yml 예시

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: mypassword
```

```
    ports:
      - "5432:5432"

  redis:
    image: redis:6
    ports:
      - "6379:6379"

  kafka:
    image: confluentinc/cp-kafka:latest
    ports:
      - "9092:9092"

  app:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - postgres
      - redis
      - kafka
```

실행 명령어 단 한 줄!

```
docker-compose up -d
```

3️⃣ 깨끗한 시스템 유지

전통적인 방법의 문제

```
시간이 지나면서 컴퓨터에 쌓이는 것들:

C:\Program Files\
├── Java 8
├── Java 11
├── Java 17
├── Python 2.7
├── Python 3.8
├── Python 3.9
├── Node.js 12
├── Node.js 14
├── Node.js 16
├── PostgreSQL 12
├── PostgreSQL 13
├── MySQL 5.7
```

```
├── MySQL 8.0
└── ... (계속 쌓임)

문제점:
- 디스크 공간 낭비 (수십 GB)
- 환경 변수 충돌
- 포트 충돌
- 버전 관리 혼란
- 삭제하기 무서움 (뭘 지워야 할지 모름)
```

Docker 사용 시

```
프로젝트 종료 시:
docker-compose down -v

결과:
✅ 모든 컨테이너 삭제
✅ 모든 데이터 삭제
✅ 컴퓨터는 깨끗한 상태
✅ 필요하면 언제든 다시 시작 가능

다른 프로젝트:
docker-compose up -d (다른 설정 파일로)

결과:
✅ 완전히 독립적인 환경
✅ 서로 간섭 없음
```

4️⃣ 실제 회사들의 Docker 사용 사례

Netflix

```
문제: 수천 개의 마이크로서비스 관리
해결: Docker로 각 서비스 컨테이너화
결과: 하루 수십억 건의 API 호출 처리
```

Spotify

```
문제: 개발자들이 각자 다른 환경에서 작업
해결: Docker로 표준화된 개발 환경 제공
결과: 배포 시간 90% 단축
```

Uber

```
문제: 전 세계 여러 리전에 동일한 서비스 배포
해결: Docker 컨테이너로 일관성 보장
결과: 빠르고 안정적인 글로벌 서비스
```

💰 비용 절감 효과

서버 비용

```
전통적인 VM 방식:
- 각 애플리케이션마다 별도 VM 필요
- 1대 VM = 최소 2GB RAM, 20GB 디스크
- 10개 애플리케이션 = 10대 VM
- 월 비용: 약 $500~1000

Docker 사용:
- 하나의 서버에 여러 컨테이너 실행
- 1대 서버 = 8GB RAM으로 20~30개 컨테이너 실행
- 10개 애플리케이션 = 1~2대 서버
- 월 비용: 약 $100~200

비용 절감: 70~80% ⬇️
```

개발 시간 절약

```
개발자 시급: $50
전통적인 방법 환경 설정: 8시간 = $400
Docker 환경 설정: 0.5시간 = $25

1명당 절약: $375
10명 팀: $3,750 절약
1년에 신규 입사자 20명: $7,500 절약
```

🎯 Docker가 특히 유용한 상황

✅ 이런 경우에 Docker를 꼭 사용하세요

1. **마이크로서비스 아키텍처**

   ```
   여러 개의 작은 서비스로 구성된 시스템
   → 각 서비스를 독립적인 컨테이너로 관리
   ```

2. **팀 협업 프로젝트**

   ```
   여러 명이 같은 프로젝트 작업
   ```

```
→ 모든 팀원이 동일한 개발 환경 사용
```

3. **CI/CD 파이프라인**

```
자동 빌드, 테스트, 배포
→ 일관된 환경에서 자동화
```

4. **레거시 시스템 유지보수**

```
오래된 버전의 라이브러리 필요
→ 격리된 환경에서 안전하게 실행
```

5. **여러 프로젝트 동시 작업**

```
프로젝트 A: Java 8 + MySQL
프로젝트 B: Java 11 + PostgreSQL
→ 각각 독립적인 컨테이너로 실행
```

⚠️ Docker가 과할 수 있는 상황

1. **아주 간단한 스크립트**

```
Python 스크립트 하나만 실행
→ Docker 없이도 충분
```

2. **데스크톱 GUI 애플리케이션**

```
Photoshop, Excel 같은 프로그램
→ Docker로 실행하기 어려움
```

3. **극도로 높은 성능이 필요한 경우**

```
게임, 실시간 영상 처리 등
→ Native 실행이 더 빠름 (하지만 차이는 미미함)
```

1.3 핵심 용어 설명

Docker를 이해하기 위해 꼭 알아야 할 4가지 핵심 개념을 자세히 알아봅시다.

🏗️ 이미지 (Image)

📖 이미지란?

이미지는 컨테이너를 만들기 위한 "설계도" 또는 "템플릿"입니다.

🎯 쉬운 비유

```
이미지 = 붕어빵 틀
컨테이너 = 실제 붕어빵

🔧 붕어빵 틀 (이미지)
- 한 번 만들면 계속 사용 가능
- 똑같은 모양의 붕어빵을 여러 개 만들 수 있음
- 틀 자체는 먹을 수 없음

🍞 붕어빵 (컨테이너)
- 틀을 사용해서 만든 실제 붕어빵
- 여러 개 만들 수 있음
- 각각 독립적 (하나를 먹어도 다른 것에 영향 없음)
```

💾 이미지의 구조

레이어 시스템

```
PostgreSQL 이미지 구조:

┌─────────────────────────────┐
├─────────────────────────────┤  ⬅️ Layer 4: PostgreSQL 설정 파일 (1 MB)
├─────────────────────────────┤  ⬅️ Layer 3: PostgreSQL 프로그램 (50 MB)
├─────────────────────────────┤  ⬅️ Layer 2: 필요한 라이브러리들 (30 MB)
├─────────────────────────────┤  ⬅️ Layer 1: 기본 OS (Ubuntu 20 MB)
└─────────────────────────────┘

각 레이어는 읽기 전용 (변경 불가)
위로 쌓아 올리는 구조
```

왜 레이어로 나눌까?

```
장점 1: 효율적인 저장
- Ubuntu 레이어는 여러 이미지가 공유
- PostgreSQL, MySQL, Redis 모두 같은 Ubuntu 레이어 사용
- 디스크 공간 절약!

장점 2: 빠른 다운로드
- 이미 있는 레이어는 다운로드 안 함
- 변경된 레이어만 다운로드

장점 3: 캐싱 효과
```

```
- 빌드 시 변경되지 않은 레이어는 재사용
- 빌드 시간 단축
```

📦 이미지 이름 규칙

```
이미지 이름 구조:
[레지스트리]/[사용자명]/[이미지명]:[태그]

예시 1: postgres:13
- 레지스트리: (생략 = Docker Hub)
- 사용자명: (생략 = 공식 이미지)
- 이미지명: postgres
- 태그: 13 (버전)

예시 2: docker.io/library/postgres:13
- 레지스트리: docker.io (Docker Hub)
- 사용자명: library (공식 이미지)
- 이미지명: postgres
- 태그: 13

예시 3: mycompany/myapp:latest
- 레지스트리: (생략)
- 사용자명: mycompany
- 이미지명: myapp
- 태그: latest (최신 버전)
```

태그의 중요성

```
❌ 나쁜 예:
docker pull postgres
→ 어떤 버전인지 모름 (latest)
→ 내일 다시 받으면 다른 버전일 수 있음

✅ 좋은 예:
docker pull postgres:13.8
→ 정확한 버전 명시
→ 언제 받아도 동일한 버전
```

🔍 이미지 찾기

Docker Hub (https://hub.docker.com/)

```
Docker Hub = 이미지 저장소 (앱 스토어와 비슷)
```

```
인기 이미지:
- postgres: 10억+ 다운로드
- nginx: 50억+ 다운로드
- redis: 10억+ 다운로드
- node: 20억+ 다운로드

각 이미지 페이지에서 확인 가능:
✅ 사용법 (README)
✅ 지원하는 태그 (버전)
✅ Dockerfile 소스
✅ 보안 스캔 결과
```

💻 이미지 관련 명령어

```bash
# 이미지 검색
docker search postgres

# 이미지 다운로드
docker pull postgres:13

# 다운로드한 이미지 목록 보기
docker images

# 출력 예시:
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
postgres      13        a1b2c3d4e5f6   2 weeks ago     374MB
redis         6         b2c3d4e5f6a1   1 month ago     105MB

# 이미지 상세 정보
docker inspect postgres:13

# 이미지 삭제
docker rmi postgres:13

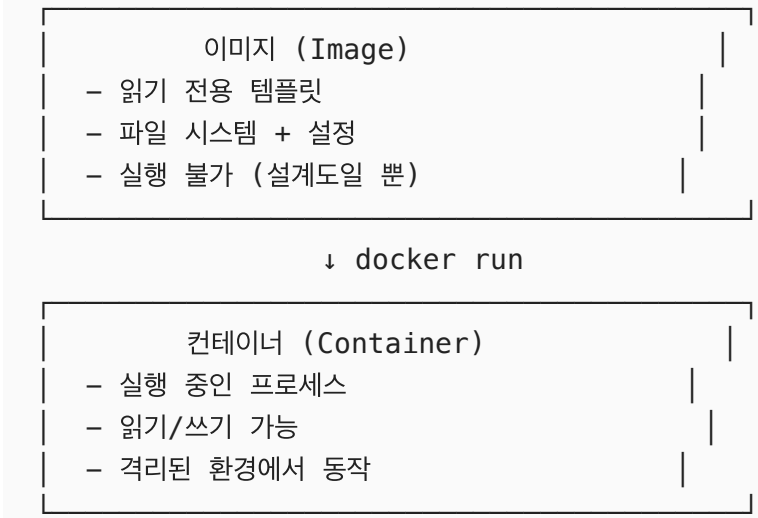
# 사용하지 않는 이미지 모두 삭제
docker image prune -a
```

📦 컨테이너 (Container)

📖 컨테이너란?

컨테이너는 이미지를 실제로 실행한 것입니다. 격리된 환경에서 독립적으로 동작하는 프로세스입니다.

🎯 이미지 vs 컨테이너



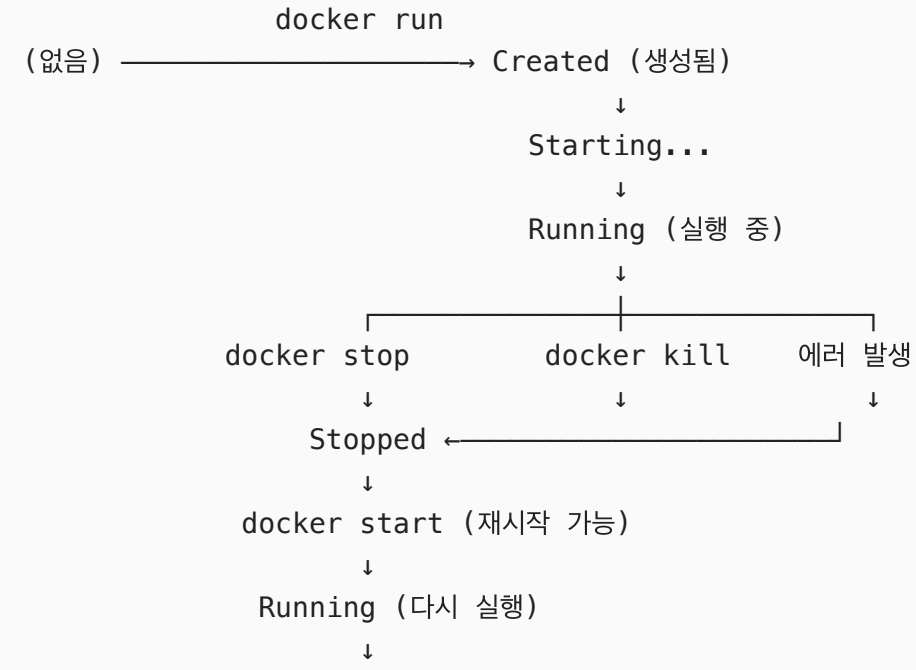
실제 예시

```
# 이미지 하나로 여러 컨테이너 생성 가능

postgres:13 이미지
    ↓
├── 컨테이너 1: 개발용 DB (포트 5432)
├── 컨테이너 2: 테스트용 DB (포트 5433)
└── 컨테이너 3: 스테이징 DB (포트 5434)

각각 독립적으로 실행되며 서로 간섭하지 않음!
```

🔄 컨테이너 생명주기



```
                    docker rm (삭제)
                          ↓
                    (제거됨)
```

상태별 설명

```
Created (생성됨):
- 이미지로부터 컨테이너 생성 완료
- 아직 실행되지 않음
- 설정만 완료된 상태

Running (실행 중):
- 컨테이너가 실제로 동작 중
- 프로세스가 실행되고 있음
- CPU, 메모리 사용 중

Stopped (중지됨):
- 실행이 중지됨
- 데이터는 그대로 보존
- 재시작 가능

Removed (제거됨):
- 컨테이너 완전 삭제
- 데이터도 함께 삭제 (볼륨 제외)
- 복구 불가능
```

💻 컨테이너 관련 명령어

```
# 컨테이너 실행 (이미지 → 컨테이너)
docker run --name mypostgres -e POSTGRES_PASSWORD=mysecret -p 5432:5432 -d
postgres:13

명령어 분석:
--name mypostgres      → 컨테이너 이름 지정
-e POSTGRES_PASSWORD   → 환경 변수 설정
-p 5432:5432           → 포트 매핑 (호스트:컨테이너)
-d                     → 백그라운드 실행 (detached)
postgres:13            → 사용할 이미지

# 실행 중인 컨테이너 목록
docker ps

# 출력 예시:
CONTAINER ID   IMAGE         COMMAND                  STATUS         PORTS
NAMES
```

```
a1b2c3d4e5f6   postgres:13   "docker-entrypoint..."   Up 2 hours
0.0.0.0:5432->5432/tcp   mypostgres

# 모든 컨테이너 목록 (중지된 것 포함)
docker ps -a

# 컨테이너 중지
docker stop mypostgres

# 컨테이너 시작 (중지된 컨테이너 재시작)
docker start mypostgres

# 컨테이너 재시작
docker restart mypostgres

# 컨테이너 삭제 (중지 상태여야 함)
docker rm mypostgres

# 실행 중인 컨테이너 강제 삭제
docker rm -f mypostgres

# 컨테이너 로그 확인
docker logs mypostgres

# 실시간 로그 확인
docker logs -f mypostgres

# 컨테이너 내부 접속
docker exec -it mypostgres bash

# 컨테이너 상세 정보
docker inspect mypostgres

# 컨테이너 리소스 사용량 확인
docker stats mypostgres

# 모든 중지된 컨테이너 삭제
docker container prune
```

🔧 컨테이너 격리의 의미

```
호스트 OS (내 컴퓨터)
├── 컨테이너 A
│   ├── 프로세스 ID: 1, 2, 3 (컨테이너 내부 관점)
│   ├── 파일 시스템: /app, /data
```

```
│    ├── 네트워크: 172.17.0.2
│    └── 포트: 8080
│
├── 컨테이너 B
│    ├── 프로세스 ID: 1, 2, 3 (또 다른 독립적인 번호!)
│    ├── 파일 시스템: /app, /data (A와 다른 공간)
│    ├── 네트워크: 172.17.0.3
│    └── 포트: 8080 (A와 충돌하지 않음!)
│
└── 컨테이너 C
     ├── ...

각 컨테이너는 마치 독립된 컴퓨터처럼 동작!
```

🎼 Docker Compose

📖 Docker Compose란?

Docker Compose는 여러 개의 컨테이너를 하나의 서비스처럼 관리할 수 있게 해주는 도구입니다.

🤔 왜 필요한가?

문제 상황: 복잡한 애플리케이션

```
웹 애플리케이션을 실행하려면:
1. PostgreSQL 컨테이너 실행
2. Redis 컨테이너 실행
3. Kafka 컨테이너 실행
4. Zookeeper 컨테이너 실행
5. 애플리케이션 컨테이너 실행

각각 docker run 명령어로 실행해야 함...
```

명령어가 너무 복잡해짐

```
# PostgreSQL 실행
docker run --name postgres -e POSTGRES_PASSWORD=mysecret -p 5432:5432 -d
postgres:13

# Redis 실행
docker run --name redis -p 6379:6379 -d redis:6

# Kafka 실행 (Zookeeper도 필요)
docker run --name zookeeper -p 2181:2181 -d confluentinc/cp-zookeeper:latest
docker run --name kafka --link zookeeper -p 9092:9092 -e
```

```
KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 -d confluentinc/cp-kafka:latest

# 애플리케이션 실행
docker run --name myapp --link postgres --link redis --link kafka -p
8080:8080 -d myapp:latest

총 5개 명령어!
순서도 중요!
옵션도 복잡!
```

Docker Compose 사용 시

```
# docker-compose.yml 파일 하나로 모든 것 정의

version: '3.8'

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: mysecret
    ports:
      - "5432:5432"

  redis:
    image: redis:6
    ports:
      - "6379:6379"

  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"

  app:
    image: myapp:latest
    depends_on:
      - postgres
      - redis
```

```
      - kafka
    ports:
      - "8080:8080"
```

실행 명령어 단 한 줄!

```
# 모든 서비스 시작
docker-compose up -d

# 모든 서비스 중지 및 삭제
docker-compose down
```

🎯 Docker Compose의 장점

1. 선언적 설정

```
명령형 (docker run):
"이렇게 해라, 저렇게 해라" → 절차적 명령

선언형 (docker-compose.yml):
"이런 상태가 되어야 한다" → 최종 상태 선언

장점:
- 읽기 쉬움
- 이해하기 쉬움
- 버전 관리 가능 (Git으로 관리)
- 팀원과 공유 쉬움
```

2. 의존성 관리

```
services:
  app:
    depends_on:
      - postgres
      - redis

의미:
- postgres와 redis가 먼저 시작
- 그 다음에 app 시작
- 순서 자동 관리!
```

3. 네트워크 자동 생성

```
Docker Compose는 자동으로 네트워크 생성

같은 docker-compose.yml 안의 서비스들은:
✅ 서비스 이름으로 서로 통신 가능
✅ 별도 설정 불필요

예시:
app 컨테이너에서 postgres 접속:
호스트명: postgres (그냥 서비스 이름 사용!)
포트: 5432
```

4. 환경별 설정 오버라이드

```
기본 설정: docker-compose.yml
개발 환경: docker-compose.override.yml
운영 환경: docker-compose.prod.yml

실행:
# 개발 환경 (기본 + override 자동 병합)
docker-compose up

# 운영 환경
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up
```

💻 Docker Compose 명령어

```
# 서비스 시작 (백그라운드)
docker-compose up -d

# 서비스 시작 (로그 보면서)
docker-compose up

# 특정 서비스만 시작
docker-compose up -d postgres redis

# 서비스 중지 (컨테이너 유지)
docker-compose stop

# 서비스 시작 (중지된 컨테이너 재시작)
docker-compose start

# 서비스 중지 및 컨테이너 삭제
docker-compose down
```

```
# 서비스 중지 및 볼륨도 삭제
docker-compose down -v

# 실행 중인 서비스 목록
docker-compose ps

# 서비스 로그 확인
docker-compose logs

# 특정 서비스 로그
docker-compose logs postgres

# 실시간 로그
docker-compose logs -f

# 서비스 재시작
docker-compose restart

# 특정 서비스 재시작
docker-compose restart postgres

# 서비스 내부 명령 실행
docker-compose exec postgres psql -U postgres

# 서비스 스케일링 (같은 서비스 여러 개 실행)
docker-compose up -d --scale app=3
```

📋 docker-compose.yml 기본 구조

```
version: '3.8'  # Docker Compose 파일 버전

services:       # 실행할 서비스들 정의

  service1:     # 첫 번째 서비스
    image: postgres:13              # 사용할 이미지
    container_name: my-postgres     # 컨테이너 이름 (선택)

    environment:                    # 환경 변수
      POSTGRES_PASSWORD: secret
      POSTGRES_USER: admin

    ports:                          # 포트 매핑
      - "5432:5432"                 # 호스트:컨테이너

    volumes:                        # 볼륨 마운트
```

```
      - postgres-data:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

    networks:                       # 네트워크 연결
      - mynetwork

    restart: unless-stopped         # 재시작 정책

    depends_on:                     # 의존성
      - service2

  service2:     # 두 번째 서비스
    image: redis:6
    ports:
      - "6379:6379"

volumes:        # 볼륨 정의
  postgres-data:
    driver: local

networks:       # 네트워크 정의
  mynetwork:
    driver: bridge
```

💾 볼륨 (Volume)

📖 볼륨이란?

볼륨은 컨테이너의 데이터를 영구적으로 저장하는 공간입니다.

🤔 왜 필요한가?

문제: 컨테이너는 임시적

```
컨테이너의 특성:
- 컨테이너를 삭제하면 내부 데이터도 함께 삭제됨
- 컨테이너를 재시작하면 데이터는 유지되지만...
- 재생성하면 모든 데이터 사라짐!

예시:
1. PostgreSQL 컨테이너 실행
2. 데이터베이스에 데이터 입력
3. 컨테이너 삭제 (docker rm)
4. 다시 컨테이너 실행
5. 데이터가 모두 사라짐! 😱
```

해결: 볼륨 사용

```
볼륨을 사용하면:
✅ 컨테이너를 삭제해도 데이터 보존
✅ 새 컨테이너에서 기존 데이터 사용 가능
✅ 여러 컨테이너가 같은 데이터 공유 가능
✅ 백업과 복원 쉬움
```

🎯 볼륨의 종류

1. Named Volume (이름 있는 볼륨)

```
# 볼륨 생성
docker volume create postgres-data

# 컨테이너에서 볼륨 사용
docker run -v postgres-data:/var/lib/postgresql/data postgres:13

특징:
✅ Docker가 관리
✅ 위치 신경 쓸 필요 없음
✅ 가장 권장되는 방법
```

2. Bind Mount (경로 직접 지정)

```
# 내 컴퓨터의 특정 폴더를 컨테이너에 연결
docker run -v /Users/myname/data:/var/lib/postgresql/data postgres:13

특징:
✅ 정확한 위치 지정 가능
✅ 개발 중 코드 변경 즉시 반영 (Hot Reload)
⚠️ 경로 관리 필요
⚠️ OS마다 경로 다름
```

3. Anonymous Volume (익명 볼륨)

```
# 이름 없이 자동 생성
docker run -v /var/lib/postgresql/data postgres:13

특징:
⚠️ 이름이 랜덤 생성
⚠️ 관리하기 어려움
⚠️ 거의 사용하지 않음
```

📊 볼륨 비교

특징	Named Volume	Bind Mount	Anonymous Vol
Docker 관리	✅	❌	✅
경로 지정	❌	✅	❌
백업 쉬움	✅	✅	❌
성능	빠름	빠름	빠름
추천도	⭐⭐⭐	⭐⭐	⭐

💻 볼륨 관련 명령어

```
# 볼륨 목록 보기
docker volume ls

# 출력 예시:
DRIVER    VOLUME NAME
local     postgres-data
local     redis-data

# 볼륨 생성
docker volume create my-volume

# 볼륨 상세 정보
docker volume inspect my-volume

# 출력 예시:
[
    {
        "CreatedAt": "2024-01-15T10:00:00Z",
        "Driver": "local",
        "Mountpoint": "/var/lib/docker/volumes/my-volume/_data",
        "Name": "my-volume"
    }
]

# 볼륨 삭제
docker volume rm my-volume

# 사용하지 않는 볼륨 모두 삭제
docker volume prune

# 경고와 함께 확인 후 삭제
```

```
docker volume prune
WARNING! This will remove all local volumes not used by at least one
container.
Are you sure you want to continue? [y/N] y
```

🔧 실전 예시

PostgreSQL 데이터 영구 보존

```
# 1. 볼륨 생성
docker volume create postgres-data

# 2. 컨테이너 실행 (볼륨 연결)
docker run --name mydb \
  -e POSTGRES_PASSWORD=secret \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  -d postgres:13

# 3. 데이터 입력
docker exec -it mydb psql -U postgres
postgres=# CREATE TABLE users (id INT, name VARCHAR(50));
postgres=# INSERT INTO users VALUES (1, 'Alice');
postgres=# \q

# 4. 컨테이너 삭제
docker rm -f mydb

# 5. 새 컨테이너로 재시작 (같은 볼륨 사용)
docker run --name mydb2 \
  -e POSTGRES_PASSWORD=secret \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  -d postgres:13

# 6. 데이터 확인 (그대로 남아있음!)
docker exec -it mydb2 psql -U postgres
postgres=# SELECT * FROM users;
 id | name
----+-------
  1 | Alice
(1 row)
```

개발 중 코드 Hot Reload (Bind Mount)

```
# 프로젝트 폴더를 컨테이너에 마운트
docker run --name myapp \
  -v /Users/myname/project:/app \
  -p 3000:3000 \
  node:16

# 이제 로컬에서 코드 수정하면
# 컨테이너 안에서도 즉시 반영됨!
```

Docker Compose에서 볼륨 사용

```
version: '3.8'

services:
  postgres:
    image: postgres:13
    volumes:
      # Named Volume
      - postgres-data:/var/lib/postgresql/data
      # Bind Mount (초기화 스크립트)
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5432:5432"

  app:
    image: myapp:latest
    volumes:
      # 개발 중 코드 Hot Reload
      - ./src:/app/src
    ports:
      - "8080:8080"

volumes:
  postgres-data:    # Named Volume 정의
    driver: local
```

📂 볼륨 저장 위치

```
운영체제별 Docker 볼륨 위치:

Linux:
/var/lib/docker/volumes/

Mac (Docker Desktop):
```

```
~/Library/Containers/com.docker.docker/Data/vms/0/

Windows (Docker Desktop):
\\wsl$\docker-desktop-data\version-pack-data\community\docker\volumes\

⚠️ 직접 접근보다는 Docker 명령어 사용 권장!
```

💡 볼륨 사용 베스트 프랙티스

```
1. 데이터베이스는 항상 Named Volume 사용
   docker-compose.yml:
   volumes:
     - postgres-data:/var/lib/postgresql/data

2. 개발 중 소스 코드는 Bind Mount
   volumes:
     - ./src:/app/src

3. 로그 파일은 Named Volume
   volumes:
     - app-logs:/var/log/app

4. 임시 파일은 tmpfs (메모리 사용)
   tmpfs:
     - /tmp

5. 정기적 백업 설정
   docker run --rm -v postgres-data:/data -v $(pwd):/backup \
     alpine tar czf /backup/backup.tar.gz /data
```

📊 수치로 보는 효과

개발 생산성 비교

지표	Docker 없이	Docker 사용	개선율
개발 환경 설정 시간	4-8시간	5-10분	**95% ↓**
신규 팀원 온보딩	1-2일	10-15분	**95% ↓**
프로젝트 전환 시간	2-4시간	1-2분	**98% ↓**
배포 시간	30-60분	2-5분	**90% ↓**

비용 절감 효과

항목	전통적 방법	Docker 사용	절감율
서버 비용 (월)	$500-1,000	$100-200	**70-80% ↓**
개발자 환경설정 비용	$400/명	$25/명	**94% ↓**
인프라 관리 시간	40시간/월	8시간/월	**80% ↓**
장애 복구 시간	2-4시간	10-30분	**85% ↓**

성능 비교

지표	가상머신 (VM)	Docker 컨테이너	개선
시작 시간	수 분	수 초	**95% ↓**
메모리 사용량	수 GB	수십 MB	**95% ↓**
디스크 사용량	수십 GB	수백 MB	**90% ↓**
동시 실행 개수	5-10개	50-100개	**10배 ↑**

실제 기업 효과

기업	적용 전	적용 후	성과
Spotify	배포 1시간	배포 6분	**90% 단축**
PayPal	하루 6회 배포	하루 1,000회	**166배 증가**
Netflix	수동 배포	자동 배포	**하루 수천 회**
ING Bank	배포 6주	배포 1일	**97% 단축**

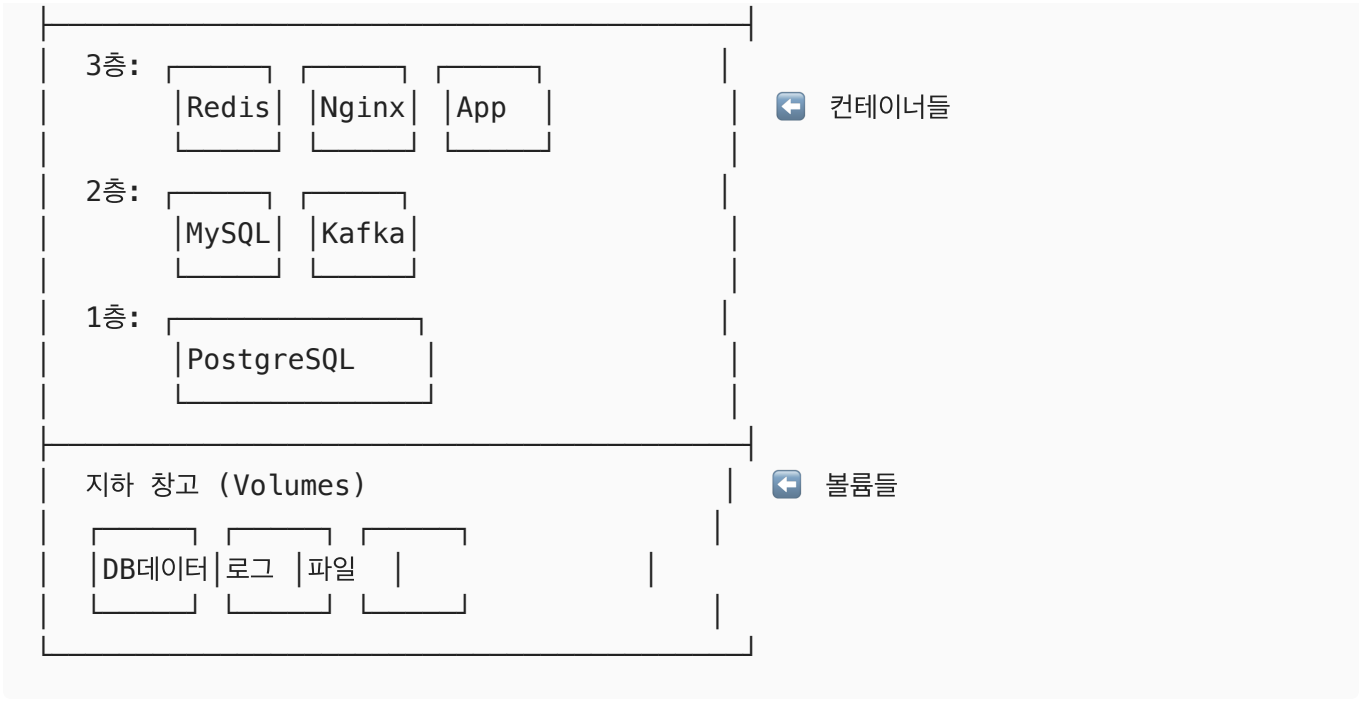
🌟 실생활 비유로 이해하기

Docker의 개념을 실생활 비유로 완벽하게 이해해봅시다!

🏢 비유 1: 아파트 건물

```
Docker = 아파트 건물 관리 시스템

┌─────────────────────────────────────────┐
│        아파트 건물 (Docker Host)          │
```



각 요소 매칭:

```
이미지 (Image) = 아파트 설계도면
- 같은 설계도로 여러 호실 만들 수 있음
- 설계도는 변경 불가 (읽기 전용)

컨테이너 (Container) = 실제 아파트 호실
- 각 호실은 독립적 (101호와 102호는 별개)
- 입주자는 자유롭게 꾸밀 수 있음
- 퇴거하면 원상복구 (삭제하면 초기화)

볼륨 (Volume) = 지하 창고
- 개인 창고는 퇴거해도 그대로 유지
- 새로운 호실로 이사해도 창고 내용물 보존
- 여러 호실이 같은 창고 공유 가능

Docker Compose = 아파트 관리사무소
- 여러 호실을 한 번에 관리
- 전기, 수도, 난방 일괄 제어
- 엘리베이터, 복도 등 공용 시설 관리

네트워크 (Network) = 복도와 엘리베이터
- 같은 층 호실들끼리 소통 가능
- 다른 층으로도 이동 가능 (엘리베이터)
- 외부 손님 방문 (포트 개방)
```

실제 시나리오:

```
입주 (컨테이너 실행):
docker run postgres:13
→ "PostgreSQL 호실" 입주 시작

퇴거 (컨테이너 삭제):
docker rm postgres
→ 호실 비우고 퇴거 (창고 내용물은 남음)

리모델링 (이미지 업데이트):
docker pull postgres:14
→ 새로운 설계도 다운로드

이사 (볼륨 재사용):
docker run -v old-data:/var/lib/postgresql/data postgres:14
→ 새 호실로 이사했지만 창고는 그대로 사용
```

🍔 비유 2: 패스트푸드 프랜차이즈

```
Docker = 맥도날드 프랜차이즈 시스템

본사 (Docker Hub)
├── 햄버거 레시피 (Image)
├── 매장 인테리어 설계도 (Dockerfile)
└── 운영 매뉴얼 (docker-compose.yml)

지역 매장들 (Containers)
├── 서울 강남점 (Container 1)
├── 부산 해운대점 (Container 2)
└── 대구 동성로점 (Container 3)

각 매장은:
✅ 동일한 레시피 사용 (이미지)
✅ 같은 인테리어 (설정)
✅ 독립적으로 운영 (격리)
✅ 언제든 폐점/개점 가능 (생성/삭제)
```

시나리오:

```
신규 매장 오픈:
docker run mcdonalds:latest
→ 레시피대로 빠르게 매장 오픈

동일한 품질:
서울에서 먹는 빅맥 = 부산에서 먹는 빅맥
```

```
→ 어디서 실행해도 동일한 결과

재료 창고:
volumes:
  - ingredients:/storage
→ 매장이 문 닫아도 재료는 보관

메뉴 업데이트:
docker pull mcdonalds:2024
→ 새로운 메뉴 레시피 배포
```

🚢 비유 3: 화물선 운송

```
Docker = 표준 컨테이너 운송 시스템


┌─────────────────────────────────────┐
│      화물선 (Docker Host)           │
├─────────────────────────────────────┤
│                                     │
│   ┌──┐ ┌──┐ ┌──┐ ┌──┐             │
│   │바나나│ │전자│ │의류│ │가구│      ⬅️ 컨테이너들
│   └──┘ └──┘ └──┘ └──┘             │
│                                     │
│   ┌──┐ ┌──┐ ┌──┐                  │
│   │식품│ │자동차│ │화학│             │
│   └──┘ └──┘ └──┘                  │
└─────────────────────────────────────┘

모든 컨테이너는 표준 규격 (20ft, 40ft)
→ 어떤 배, 기차, 트럭에도 적재 가능!
```

장점:

```
1. 표준화:
   전 세계 어느 항구에서든 동일하게 처리
   = Docker 컨테이너는 어느 서버에서든 실행

2. 효율성:
   빠른 적재/하역
   = Docker는 빠른 시작/중지

3. 독립성:
   바나나 컨테이너와 화학물질 컨테이너 분리
   = 각 서비스가 서로 간섭하지 않음

4. 추적 가능:
```

```
    컨테이너 번호로 위치 추적
    = docker ps로 상태 확인
```

🎮 비유 4: 게임 세이브 파일

```
Docker = 게임 세이브 시스템

게임 디스크 (Image):
- 게임 설치 파일
- 읽기 전용
- 복사해서 여러 번 설치 가능

게임 실행 (Container):
- 실제 게임 플레이
- 캐릭터 움직임, 전투 등
- 종료하면 프로세스 종료

세이브 파일 (Volume):
- 게임 진행 상황 저장
- 레벨, 아이템, 경험치
- 게임 재설치해도 세이브 파일은 보존
```

시나리오:

```
게임 시작:
docker run game:latest
→ 새 게임 시작

진행 상황 저장:
-v save-data:/game/saves
→ 세이브 파일 보관

게임 업데이트:
docker pull game:v2.0
docker run -v save-data:/game/saves game:v2.0
→ 새 버전으로 업데이트했지만 세이브는 그대로

게임 종료:
docker stop game
→ 게임 종료 (세이브는 보존)

게임 삭제:
docker rm game
→ 게임 제거 (세이브는 보존)
```

```
완전 삭제:
docker rm game && docker volume rm save-data
→ 세이브까지 완전 삭제
```

📚 비유 5: 도서관 시스템

```
Docker = 도서관 관리 시스템

원본 도서 (Image):
- 출판사에서 인쇄된 책
- 여러 권 복사 가능
- 원본은 수정 불가

대출 도서 (Container):
- 독자가 빌려간 책
- 메모, 밑줄 가능 (쓰기 가능 레이어)
- 반납하면 원상복구

독서 노트 (Volume):
- 독자의 독서 기록
- 책을 반납해도 노트는 남음
- 다른 책 빌려도 노트 계속 사용

사서 (Docker Compose):
- 여러 권을 세트로 관리
- "파이썬 시리즈 전권 대출"
- 일괄 대출/반납
```

🏭 비유 6: 공장 생산라인

```
Docker = 모듈식 생산 공장

설계도 (Dockerfile):
FROM basemodel:latest        ← 기본 프레임
RUN install engine           ← 엔진 설치
COPY parts /assembly         ← 부품 조립
CMD ["start", "production"] ← 생산 시작

생산라인 (Docker Compose):
services:
  부품제조:
    image: parts-maker
  조립:
    image: assembler
```

```
    depends_on:
      - 부품제조
  포장:
    image: packaging
    depends_on:
      - 조립

각 공정은 독립적이지만 순서대로 진행!
```

🎬 종합 비교표

```
┌─────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Docker      │ 아파트       │ 패스트푸드   │ 화물선       │ 게임         │
├─────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Image       │ 설계도면     │ 레시피       │ 규격         │ 게임 디스크  │
│ Container   │ 호실         │ 매장         │ 컨테이너     │ 게임 실행    │
│ Volume      │ 창고         │ 재료창고     │ 보관소       │ 세이브파일   │
│ Compose     │ 관리소       │ 본사         │ 운송회사     │ 세트 판매    │
│ Network     │ 복도         │ 배달망       │ 운송경로     │ 멀티플레이   │
└─────────────┴──────────────┴──────────────┴──────────────┴──────────────┘
```

💡 핵심 포인트

이 모든 비유에서 공통적인 것:

```
1. 표준화 (Standardization)
   → 한 번 만들면 어디서든 동일하게 작동

2. 격리 (Isolation)
   → 각각 독립적으로 실행, 서로 간섭 없음

3. 재사용성 (Reusability)
   → 같은 템플릿으로 여러 인스턴스 생성

4. 이식성 (Portability)
   → 환경에 관계없이 동일한 결과

5. 효율성 (Efficiency)
   → 빠른 시작, 적은 리소스 사용
```

👨‍💻 주니어 개발자 시나리오

시나리오 1: 첫 출근날, 환경 설정으로 하루가 끝났다

상황: 첫 출근날, 선배가 프로젝트 저장소 주소를 알려주며 "개발 환경 설정하고 로컬에서 실행해봐"라고 했습니다.

❌ Docker를 모르는 주니어의 하루

```
09:00 - Git 저장소 클론
git clone https://github.com/company/project.git

09:10 - README 읽기 시작
"Java 11, PostgreSQL 13, Redis 6, Kafka 2.8 필요..."

09:30 - Java 11 설치 시도
# 이미 Java 8이 설치되어 있음
# JAVA_HOME 충돌로 고생

10:30 - PostgreSQL 설치
# 포트 5432가 이미 사용 중
# 이전 프로젝트에서 설치한 PostgreSQL 9.6과 충돌

12:00 - 점심 먹으면서 Stack Overflow 검색

13:00 - PostgreSQL 삭제 후 재설치
# 기존 데이터베이스까지 날아감... 😱

14:00 - Redis 설치 시도
# Windows라서 공식 지원 안 됨
# WSL2 설치부터 시작...

16:00 - Kafka 설치 포기
# Zookeeper도 필요하다고?
# 설정이 너무 복잡...

17:30 - 선배에게 "내일 다시 해볼게요..." 😭
```

문제점:

- 환경 설정에만 하루가 소요
- 기존 설치된 프로그램과 충돌
- Windows 환경에서 일부 도구 설치 어려움
- 개발은 한 줄도 못 함

✅ Docker를 아는 주니어의 하루

```
09:00 - Git 저장소 클론
git clone https://github.com/company/project.git
cd project

09:05 - Docker Compose 실행
docker-compose up -d

09:08 - 모든 서비스 자동 실행 완료!
Creating network "project_default" with the default driver
Creating project_postgres_1 ... done
Creating project_redis_1    ... done
Creating project_kafka_1    ... done
Creating project_app_1      ... done

09:10 - 브라우저로 접속 확인
http://localhost:8080
"Welcome to Company Project!" ✅

09:15 - 코드 수정 시작
# 실제 개발 업무 시작!

12:00 - 점심 먹으면서 첫 PR 작성

17:00 - 선배: "벌써 끝냈어? 대단한데?" 😊
```

배운 점:

- Docker Compose 하나로 전체 환경 구성
- 시간 절약: 8시간 → 10분
- 선배의 신뢰 획득
- 실제 업무에 집중 가능

시나리오 2: "제 컴퓨터에서는 되는데요?" 문제

상황: 로컬에서 완벽하게 작동하는 코드를 서버에 배포했는데 안 됩니다.

❌ 주니어가 작성한 코드

```
// 로컬 환경에서 개발
public class DatabaseConfig {
    private static final String DB_URL =
"jdbc:postgresql://localhost:5432/mydb";
```

```
    private static final String DB_USER = "postgres";
    private static final String DB_PASSWORD = "1234";

    public Connection getConnection() {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
}
```

문제 발생:

```
로컬 (Windows):
- PostgreSQL 13.5
- 포트: 5432
- 타임존: Asia/Seoul
→ 정상 작동 ✅

개발 서버 (Linux):
- PostgreSQL 12.8 (버전 다름!)
- 포트: 5432
- 타임존: UTC
→ 날짜 관련 버그 발생 ❌

운영 서버 (Linux):
- PostgreSQL 14.2 (또 다른 버전!)
- 포트: 5433 (포트 다름!)
- 타임존: UTC
→ 연결 자체가 안 됨 ❌

Slack 메시지:
팀장: "배포 롤백하세요. 서비스 장애입니다."
주니어: "제 컴퓨터에서는 되는데요..." 😱
```

✅ Docker를 사용한 해결

```yaml
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:13.5  # 정확한 버전 명시
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 1234
      TZ: Asia/Seoul  # 타임존 명시
```

```
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data

  app:
    build: .
    environment:
      DB_URL: jdbc:postgresql://postgres:5432/mydb
      DB_USER: postgres
      DB_PASSWORD: 1234
    depends_on:
      - postgres
    ports:
      - "8080:8080"

volumes:
  postgres-data:
```

```
// 환경 변수로 설정 주입
public class DatabaseConfig {
    private static final String DB_URL = System.getenv("DB_URL");
    private static final String DB_USER = System.getenv("DB_USER");
    private static final String DB_PASSWORD = System.getenv("DB_PASSWORD");

    public Connection getConnection() {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
}
```

결과:

```
모든 환경에서 동일한 Docker 이미지 사용:
- 로컬 개발 ✅
- 개발 서버 ✅
- 스테이징 서버 ✅
- 운영 서버 ✅

Slack 메시지:
팀장: "배포 성공! 잘했어요."
주니어: "Docker 덕분입니다!" 😊
```

배운 점:

- 버전을 정확히 명시해야 함
- 환경 변수로 설정 분리
- 어디서나 동일한 환경 보장

시나리오 3: 여러 프로젝트 동시 작업

상황: 프로젝트 A 작업 중인데, 긴급하게 프로젝트 B의 버그를 수정해야 합니다.

❌ Docker 없이 프로젝트 전환

```
# 현재 프로젝트 A 작업 중
# Java 11, MySQL 8.0, Redis 6 사용 중

# 프로젝트 B로 전환 필요
# Java 8, PostgreSQL 13, Redis 5 필요

14:00 - 프로젝트 B 긴급 버그 수정 요청

14:10 - Java 버전 변경 시도
# JAVA_HOME 변경
# 환경 변수 수정
# IDE 재설정

14:40 - MySQL 중지, PostgreSQL 시작
# 포트 충돌 해결
# 기존 MySQL 백업

15:20 - Redis 버전 문제
# Redis 6 제거
# Redis 5 설치

16:00 - 드디어 프로젝트 B 실행
# 버그 수정 시작

17:30 - 다시 프로젝트 A로 복귀해야 함
# 또 환경 설정 변경... 😭
# "내일 하겠습니다..."
```

✅ Docker로 프로젝트 전환

```
# 프로젝트 A 작업 중
cd ~/projects/project-a
```

```
docker-compose up -d
# Java 11, MySQL 8.0, Redis 6 실행 중

14:00 - 프로젝트 B 긴급 버그 수정 요청

14:05 - 프로젝트 A 일시 정지
docker-compose stop
# 1초 만에 중지

14:06 - 프로젝트 B 시작
cd ~/projects/project-b
docker-compose up -d
# Java 8, PostgreSQL 13, Redis 5 자동 실행

14:08 - 프로젝트 B 실행 완료!
# 버그 수정 시작

16:00 - 버그 수정 완료, 프로젝트 A로 복귀

16:01 - 프로젝트 B 중지
docker-compose stop

16:02 - 프로젝트 A 재시작
cd ~/projects/project-a
docker-compose start

16:04 - 이전 작업 이어서 진행
# 모든 데이터 그대로 유지 ✅
```

시간 비교:

```
전통적 방법:
- 프로젝트 전환: 2시간
- 다시 복귀: 1시간
- 총 낭비 시간: 3시간

Docker 사용:
- 프로젝트 전환: 2분
- 다시 복귀: 2분
- 총 소요 시간: 4분

시간 절약: 176분 (약 3시간!)
```

배운 점:

- 여러 프로젝트를 독립적으로 관리
- 빠른 전환으로 생산성 향상
- 각 프로젝트의 데이터 보존

시나리오 4: 컴퓨터 포맷 후 복구

상황: 컴퓨터가 느려져서 포맷했습니다. 개발 환경을 다시 설정해야 합니다.

❌ Docker 없이 복구

```
Day 1:
09:00 - OS 재설치 완료
10:00 - Java 설치 (버전이 뭐였더라?)
11:00 - Node.js 설치 (14? 16? 18?)
12:00 - 점심
13:00 - PostgreSQL 설치
13:30 - MySQL 설치
14:00 - Redis 설치 (Windows에서 어떻게 했더라?)
15:00 - MongoDB 설치
16:00 - IDE 설치 및 플러그인 설정
17:00 - Git 설정, SSH 키 복구

Day 2:
09:00 - 각 프로젝트별 의존성 설치
        npm install 실패... 버전 문제
        pip install 실패... Python 버전 문제
12:00 - 점심 먹으며 동료에게 "어떤 버전 쓰셨어요?" 물어봄
17:00 - 아직 개발 환경 설정 중... 😭

총 소요 시간: 2일
```

✅ Docker로 복구

```
Day 1:
09:00 - OS 재설치 완료

09:30 - Docker Desktop 설치
https://www.docker.com/products/docker-desktop/

09:40 - Git 설치 및 저장소 클론
git clone https://github.com/company/project-a.git
git clone https://github.com/company/project-b.git
```

```
git clone https://github.com/company/project-c.git

09:50 - 모든 프로젝트 실행
cd project-a && docker-compose up -d
cd project-b && docker-compose up -d
cd project-c && docker-compose up -d

10:00 - 개발 환경 복구 완료!
# 이미지 다운로드 자동 진행
# 모든 설정이 docker-compose.yml에 있음

10:30 - 정상 작동 확인 후 개발 시작 ✅

총 소요 시간: 1시간
```

복구 시간 비교:

```
전통적 방법: 2일 (16시간)
Docker 사용: 1시간

시간 절약: 93.75%
```

배운 점:

- docker-compose.yml이 환경 설정의 전부
- 버전 관리 걱정 없음
- 재난 복구가 매우 빠름
- Git만 있으면 모든 환경 복구 가능

❓ FAQ (자주 묻는 질문)

▶ **Q1: Docker는 가상머신(VM)과 어떻게 다른가요?**
▶ **Q2: Docker를 사용하면 성능이 느려지지 않나요?**
▶ **Q3: Docker 이미지는 어디에 저장되나요?**
▶ **Q4: Docker 컨테이너가 삭제되면 데이터도 사라지나요?**
▶ **Q5: Windows에서 Docker를 사용하면 느린가요?**
▶ **Q6: Docker는 보안이 안전한가요?**
▶ **Q7: Docker Compose와 Kubernetes의 차이는 무엇인가요?**

💼 면접 질문 리스트

📘 주니어/신입 개발자용

▶ **1. Docker란 무엇이며, 왜 사용하나요?**
▶ **2. Docker 이미지와 컨테이너의 차이를 설명해주세요.**
▶ **3. Docker 볼륨(Volume)은 왜 필요한가요?**
▶ **4. Docker Compose는 무엇이며 언제 사용하나요?**
▶ **5. Docker에서 포트 매핑(-p 옵션)은 무엇인가요?**
▶ **6. Docker 컨테이너 로그를 어떻게 확인하나요?**
▶ **7. Docker에서 환경 변수(-e 옵션)는 왜 사용하나요?**

📗 중급 개발자용

▶ **1. Docker 이미지의 레이어 시스템을 설명하고, 이것이 효율성에 어떻게 기여하는지 설명해주세요.**
▶ **2. Docker 네트워크 종류(bridge, host, overlay)와 각각의 사용 사례를 설명해주세요.**
▶ **3. 프로덕션 환경에서 Docker 컨테이너의 리소스(CPU, 메모리)를 제한해야 하는 이유와 방법을 설명해주세요.**
▶ **4. Docker 이미지를 경량화하는 방법들을 설명해주세요.**
▶ **5. Docker 컨테이너의 헬스 체크(Health Check)를 구현하는 방법과 중요성을 설명해주세요.**

🎉 축하합니다!

🏆 Docker 기본 개념 학습 완료!

이제 여러분은:
✅ Docker가 무엇이고 왜 필요한지 이해하고 설명할 수 있습니다
✅ 이미지, 컨테이너, 볼륨, Docker Compose의 개념을 명확히 알고 있습니다
✅ 실생활 비유를 통해 Docker의 원리를 쉽게 이해했습니다
✅ 실무에서 Docker를 사용하는 이유와 장점을 구체적인 수치로 알고 있습니다
✅ 주니어 개발자가 흔히 겪는 문제와 해결 방법을 배웠습니다
✅ 면접에서 Docker 관련 질문에 자신 있게 답변할 수 있습니다

📊 학습 성과

```
Before Docker:
- 개발 환경 설정: 4-8시간
- 팀원 온보딩: 1-2일
- "내 컴퓨터에서는 되는데요?" 문제 빈발
```

```
After Docker:
- 개발 환경 설정: 5-10분 (95% 단축)
- 팀원 온보딩: 10-15분 (95% 단축)
- 모든 환경에서 동일하게 작동 ✅
```

🎯 다음 단계

여러분은 이제 Docker의 기초를 완벽히 마스터했습니다!

다음 장에서는:

- Docker 실전 명령어와 실습
- Dockerfile 작성 방법
- Docker Compose 실전 활용
- 실제 프로젝트 배포하기

를 배우게 됩니다.

💪 복습 권장 사항

- [ ] 실습 과제 모두 완료하기
- [ ] 면접 질문 답변 연습하기
- [ ] 자신의 프로젝트에 Docker 적용해보기
- [ ] 동료에게 Docker를 설명해보기

🚀 실천 과제

오늘 바로 해보세요:

1. Docker Desktop 설치
2. `docker run hello-world` 실행
3. 간단한 Nginx 웹 서버 실행
4. 자신의 프로젝트에 docker-compose.yml 작성

1주일 안에 해보세요:

1. 팀 프로젝트에 Docker 도입 제안
2. 개발 환경을 Docker로 전환
3. 동료에게 Docker 튜토리얼 공유
4. 다음 장으로 진행하여 실습 강화

✅ 섹션 1 요약

핵심 개념 정리

```
🐋 Docker = 컨테이너 기반 가상화 플랫폼
   → 애플리케이션을 어디서든 동일하게 실행

🏗️ 이미지 (Image) = 실행 파일 (설계도)
   → 읽기 전용, 레이어 구조, 재사용 가능

📦 컨테이너 (Container) = 실행 중인 프로세스
   → 격리된 환경, 독립적 실행, 빠른 시작

🎼 Docker Compose = 여러 컨테이너 관리
   → 선언적 설정, 의존성 관리, 간편한 실행

💾 볼륨 (Volume) = 데이터 영구 저장
   → 컨테이너 삭제해도 데이터 보존
```

다음 섹션 예고

```
섹션 2: Docker 아키텍처와 내부 동작 원리
- 컨테이너 vs VM 상세 비교
- Docker의 내부 구조
- 네임스페이스와 cgroups
- 이미지 레이어 시스템 심화
```

실습 과제

섹션 1을 마쳤다면 다음을 시도해보세요:

```
# 1. Hello World 실행
docker run hello-world

# 2. 간단한 웹서버 실행
docker run -d -p 8080:80 nginx

# 3. 브라우저에서 확인
http://localhost:8080

# 4. 실행 중인 컨테이너 확인
docker ps

# 5. 컨테이너 중지 및 삭제
```

```
docker stop [컨테이너ID]
docker rm [컨테이너ID]

# 축하합니다! 첫 Docker 컨테이너를 실행했습니다! 🎉
```

다음 섹션에서 만나요! 👋